

SAAKSHI (साक्षी), the silent witness

Passive AI detection of cyber threats in one-way (data-diode) IP traffic

Technical Report — Smart India Hackathon 2026

Problem Statement ID	SIH26145
Problem Statement Title	AI-Based Detection of Cyber Threats in Unidirectional IP Traffic
Organisation	National Technical Research Organisation (NTRO)
Theme / Category	Blockchain & Cybersecurity / Software
Team name	F.R.I.E.N.D.S
Team ID	130208
Institution	Vignana Bharathi Institute of Technology, Hyderabad (CSE — AI & ML)
Repository	github.com/SIH-2026-CSM-A/26145 (measurements reproducible from git tags <code>idea-deck-v3</code> , <code>idea-deck-v4</code> , <code>idea-deck-v5</code>)
Date	September 2026

Team

Member
Battula Sitanshu (Team Leader)
Drona Venkata Abhiram
Adepu Yashashvi
Akula Shiva Kumar
Elluru Akshaya
Boinee Vineeth Simha

Institute mentor: Department of Computer Science and Engineering (AI & ML), Vignana Bharathi Institute of Technology.

Abstract

Critical-infrastructure operators copy their gateway and peering traffic into a monitoring enclave through hardware data diodes, so the enclave can see everything on the link but can never send anything back. NTRO's problem statement asks for an AI/ML pipeline that, from this one-way copy alone, detects, classifies and scores six threat classes in near real time: floods, botnet command-and-control, DGA and DNS tunnelling, malware in encrypted sessions, reconnaissance, and data exfiltration. It must work under five constraints: read-only ingest, no decryption, streaming with bounded latency, a stated throughput and a fixed alert schema. The hard part is that replies may be missing, most traffic is encrypted and the enclave is offline. A conventional detector silently loses signals it was built to rely on.

SAAKSHI is a passive, offline sensor built around a **Unidirectional Feature Contract**. Every feature is declared computable, degraded, reverse-dependent or unavailable. When the reply side is missing, each alert names the lost signal and the substitute used. Two lanes run side by side: a fast lane of one-second counters flags floods and scans in under 2 s (p99), and a flow lane scores rebuilt flows with rules, a LightGBM model and a DGA name model. Alerts carry evidence against a normal baseline and are sealed in a hash chain. On one laptop core the prototype processes 819–860 flows/s, and 3,915 flows/s (about 1

Gbps) on eight cores. The analysis opens no network socket. On an 11-minute capture of real botnet-dataset background with seven generated attacks, it reduced 20,857 flows to 10 alerts plus 2 early flags in 4 campaigns, and alerted on all 7 attacks.

Contents

1. Introduction — background, the problem, objectives, scope
2. Existing solutions and gap analysis
3. Proposed system and methodology
4. Implementation
5. Feasibility and testing
6. Results and impact
7. Conclusion, prototype status and future scope

References

Appendix A — The full alert, field by field

Appendix B — Reproducing every number

Appendix C — Links

Appendix D — Glossary

List of figures

- Fig. 1 Where SAAKSHI sits: production network, data diode, monitoring enclave
- Fig. 2 Architecture: one way in, two detection lanes
- Fig. 3 How the Unidirectional Feature Contract turns a lost signal into a named substitute
- Fig. 4 The alert hash chain and what verify-log checks
- Fig. 5 Sensor health states
- Fig. 6–10 Prototype screenshots: live view, campaigns, alert drawer, model card, tamper check
- Fig. 11 Measured multi-core scaling
- Fig. 12 Demo result: 20,857 flows to 4 campaigns
- Fig. 13 Campaign timeline for one internal host

List of tables

- Table 1 The five constraints of SIH26145 and where each is met
- Table 2 Gap analysis against existing approaches
- Table 3 The contract, per threat class
- Table 4 Alert fields
- Table 5 Stack, with the reason for each choice
- Table 6 Where each responsibility lives
- Table 7 Datasets
- Table 8 Latency
- Table 9 Throughput
- Table 10 One-way acceptance evidence (docs/ONEWAY-ACCEPTANCE.md)
- Table 11 Baseline (docs/BASELINE.md)
- Table 12 Validation per class
- Table 13 Operating points and limits
- Table 14 Feasibility
- Table 15 Risk register
- Table 16 Before and after, on the demo capture
- Table 17 Benefits
- Table 18 What is built and measured today, and what the three months to the Grand Finale complete

1. Introduction

1.1 Background

Power grids, telecom networks, banks and railways protect their core networks by watching the traffic that crosses their gateways. To make sure the monitoring system itself can never become a way into the core network, many operators copy the traffic through a **data diode**: a hardware device that lets bits travel in one direction only. The copy lands in a separate **monitoring enclave**. The design is deliberate. A compromised analytics box cannot pivot back into production, and the copy keeps a clean chain of custody for forensic use [1].

The price is that anything in the enclave must work only from what it passively observes: packet captures, flow records (NetFlow, IPFIX, sFlow) and metadata derived from them. It cannot probe a host, complete a handshake, look up a domain online, or send a block command back.

1.2 The problem, in our words

NTRO asks for an AI/ML pipeline that ingests this one-way stream and, in near real time, detects, classifies and scores six threat classes. It must output labelled alerts with confidence and evidence, shown on a dashboard. The difficulty is not running a classifier on packets. It is that **the evidence itself changes when the link is one-way**:

- **Replies may be missing.** A tap on one direction, or asymmetric routing, means the sensor sees requests without answers, or answers without requests. Any feature that compares the two sides, such as bytes out against bytes back, silently becomes wrong or zero.
- **Most traffic is encrypted.** Payload cannot be read, so detection must rest on metadata: sizes, timing, the plaintext part of the TLS handshake and DNS query names.
- **The enclave is offline.** There are no threat feeds, no online certificate checks and no network time. Updates must arrive as signed files.
- **The sensor must never transmit.** Every isolation claim has to be something an assessor can check, not a promise.

A detector that ignores these facts produces alerts that look normal but rest on signals the link never carried. That is worse than no alert, because an analyst in the enclave cannot go back and check.

1.3 The five constraints and the six threat classes

PS constraint	What it requires	How SAAKSHI meets it (section)
a. Read-only ingest	No return path, no live query, no inline block	0 network sockets in analysis, measured (5.4); seccomp receive-only capture, design (3.9)
b. No payload decryption	TLS/QUIC from metadata only	Plaintext ClientHello (JA4) and DNS fields only; QUIC by size and timing (3.4)
c. Streaming, not batch	Incremental processing, bounded latency	Fast lane < 2 s p99; flows scored ≤ 15 s after their last packet; long flows re-scored every 60 s (3.2, 5.2)
d. Defined throughput	State and demonstrate the tested rate	819–860 flows/s on one core; 3,915 flows/s on 8 cores (5.3)
e. Standard alert schema	timestamp, flow id, threat class, confidence, evidence	All five, plus observability state and substitutions (3.7, Appendix A)

Table 1 — The five constraints of SIH26145 and where each is met.

1.4 Objectives

1. Detect all six PS threat classes from a one-way copy, using only signals the copy actually carries.
2. Make every alert say what the link could and could not see, and what replaced a missing signal.
3. Alert on floods and scans within seconds, and on everything else within a stated bound.
4. State and measure throughput honestly, including how it scales across cores.
5. Keep evidence tamper-evident from alert to export, so it can support a Section 63 certificate under the Bharatiya Sakshya Adhiniyam, 2023 [9].
6. Prove, with checks anyone can re-run, that the sensor sends nothing onto the monitored link.

1.5 Scope

This report describes the complete proposed solution and, separately, what the working prototype already does and what was measured on it. Chapter 7 gives a status table so the two are never confused. The prototype is an MVP; shortlisted teams have about three months to finish before the Grand Finale. The sensor is detect-only: it never blocks, and it never replaces the analyst's judgement.

2. Existing Solutions and Gap Analysis

Four families of tools already watch network traffic. Each is good at what it was built for. None of them was built around the question this problem statement turns on: what happens to the evidence when half the conversation is missing and the sensor can never ask?

2.1 Signature intrusion detection (Suricata, Snort)

Signature engines match packets and reassembled streams against rules for known attacks. They are fast, mature and widely deployed, and a well-tuned ruleset catches known malware families reliably. Their limits in this setting are structural. A new attack with no published signature is invisible to them. Many rules depend on seeing both sides of a session, or on reassembling a stream they cannot follow when replies are missing. We ran Suricata 8.0.3 with the default ET Open ruleset (53,015 rules) on our own demo capture (Section 5.5). It raised no ET Open alert on the seven generated attacks, which carry no known signature. This is not a failing of Suricata; it is the case signatures were never meant to cover. SAAKSHI is designed to run alongside a tuned signature stack, not to replace it.

2.2 Network telemetry and SIEM (Zeek, NetFlow collectors)

Zeek and flow collectors turn traffic into rich logs, and a SIEM lets analysts write correlation rules over them. This is powerful, but the detection logic is whatever the analyst writes, and a log does not declare when a field was computed from one side only. Zeek's connection history does show that replies were missing, but the features and rules built on top of it do not say which of their inputs were lost. The missing side is left for the analyst to notice.

2.3 Machine-learning intrusion detection in research

The CTU-13 botnet dataset [2] showed how differently detectors behave on real, labelled botnet captures. Anderson and McGrew showed that encrypted malware traffic can be separated using flow metadata and handshake context, without decryption [5]. Antonakakis et al. showed that DGA-based malware can be detected from the names it queries [7]. JA4 gives a stable fingerprint of the plaintext TLS ClientHello [6]. LightGBM [3] and TreeSHAP [4] give fast, explainable tree models on CPU. These results are the foundation we build on. Most published systems, however, are trained and tested on two-way data and random splits, and they assume bidirectional flow features are always available.

2.4 Commercial network detection and response

Commercial NDR appliances combine signatures, behaviour analytics and cloud threat intelligence. Such products can depend on cloud lookups, vendor telemetry or remote updates, which an offline, one-way enclave cannot allow. Their models and evidence handling are also closed, so an assessor cannot check what an alert rests on.

2.5 Gap analysis

Existing approach	Works well for	Falls short on a one-way link because	How SAAKSHI addresses it
Signature IDS	Known attacks with published rules	No signature for new attacks; reply-dependent rules lose evidence	Behaviour against each host's own normal, plus an ML model; runs alongside signatures
Telemetry + SIEM	Rich logs, flexible queries	Detection is whatever the analyst writes; one-sided fields are not marked	Every feature has a declared one-way status; every alert names lost signals and substitutes
Research ML IDS	High scores on two-way benchmark data	Bidirectional features assumed; random splits leak	One-way feature contract; leave-one-scenario-out validation; no IPs, ports or clock time as features
Commercial NDR	Broad coverage with cloud intelligence	Cloud lookups and remote updates are not possible offline; closed evidence	Fully offline; signed offline updates; open, hash-chained evidence an assessor can verify

Table 2 — Gap analysis against existing approaches.

3. Proposed System and Methodology

3.1 The central idea

SAAKSHI treats observability as part of the result. Before a feature is used, the system knows whether this flow, on this link, can actually supply it. If it cannot, the detector either uses a declared substitute or does not claim the detection. Either way, the alert records what the link could and could not see. Everything else in the design follows from that rule: the two lanes, the evidence chain, and the sensor health states.

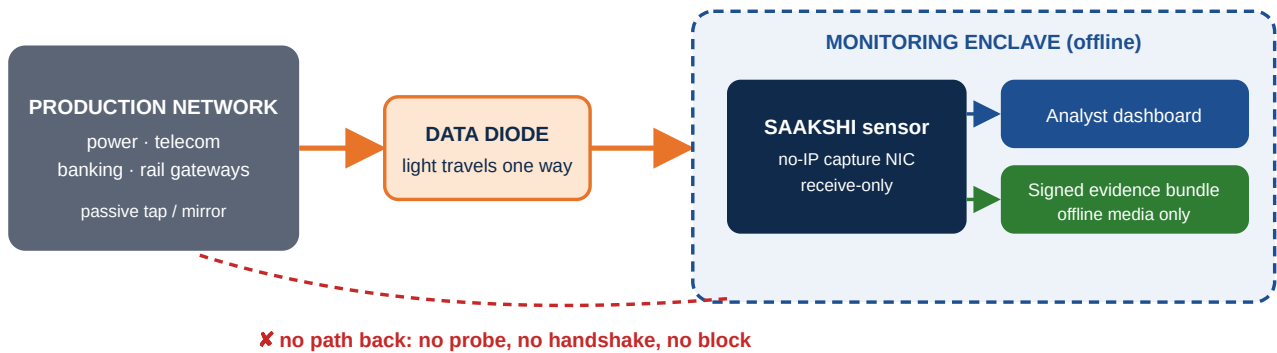


Figure 1 — Where SAAKSHI sits. Traffic is copied through the diode into the enclave; nothing flows back.

3.2 Architecture: one way in, two detection lanes

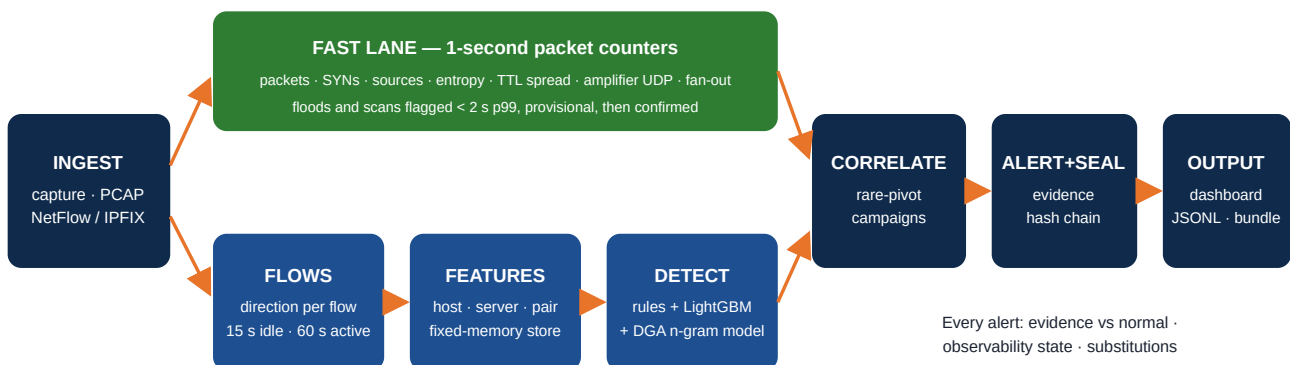


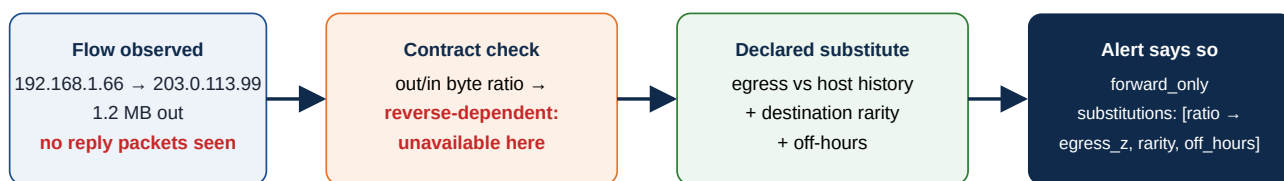
Figure 2 — Architecture. Floods and scans take the fast lane; everything else goes through flows, features and detectors. Both lanes feed correlation and a sealed alert.

- **Ingest.** The sensor reads a receive-only capture interface that has no IP address, PCAP or pcapng files, or NetFlow/IPFIX records. On a link faster than one sensor, a receive-only packet broker splits the feed. The split is keyed by internal host, so each host's history stays in one place; very busy hosts (NAT gateways, resolvers, proxies) are split further by host–server pair, and their per-host counters are merged.
- **Fast lane.** One-second counters catch volume attacks as they happen. A flood or scan raises one provisional alert per target per five minutes, and the flow lane confirms it later. This is what lets a SYN flood appear on the dashboard in under two seconds.
- **Flow lane.** Packets are rebuilt into flows, with the direction of each flow recorded. A flow is scored when it has been idle for 15 s. Long flows are re-scored every 60 s, so a slow upload is not hidden until it ends. A fixed-memory feature store keeps behaviour per host, per server and per host–server pair, using sketches, entropy and baselines, so memory stays bounded however long the sensor runs.

- **Detect.** Behaviour rules cover all six classes; a LightGBM flow model and a DGA name model add learned scores (3.5).
- **Correlate.** Alerts that share a rare pivot (a host, server, JA4 fingerprint or domain) join one campaign. Shared infrastructure, such as a public resolver or a CDN, is refused as a pivot so unrelated hosts are not merged. Each host's observed MITRE ATT&CK stages [8] are listed as history, never as a prediction.
- **Alert and seal.** Every alert is written with its evidence into a hash chain (3.8).

3.3 The Unidirectional Feature Contract

The contract is a versioned file that lists every engineered feature and its status on each kind of observation: computable, degraded, reverse-dependent or unavailable. The status is decided per flow, from what was actually captured. When a feature is unavailable, the detector uses the substitute the contract declares, and the alert records the swap (Figure 3).



The same upload with both directions captured uses the ratio directly (280 against a threshold of 50) and needs no substitute.

Figure 3 — How the contract turns a lost signal into a named substitute. This is a real alert from the prototype (Appendix A).

PS threat	One-way signal we use	When replies are missing	When only flow records exist
a. DDoS	rate, SYN share, source & TTL spread, source entropy; amplifier-port UDP against the server's norm; slow-hold (Slowloris) counting	usable whenever the attack packets cross the tap	rate, SYN share, sources
b. C2 beacon	median/MAD inter-arrival and size consistency per host–server pair (keyed by SNI or domain when visible); destination rarity	either observed direction is enough; C2 even fires on server replies alone	start times and sizes, minus exporter timeouts
c. DGA / tunnel	character n-grams, label length, record type, volume per registered domain [7]; the resolver is named	query side only	needs DNS logs
d. Encrypted	JA4 fingerprint of the plaintext ClientHello [6], new for this host, plus first-packet size and timing sequence [5]	client side only	total bytes and duration; no JA4
e. Recon	fan-out to ports and hosts over fast and slow windows; internal origins ranked first	fan-out plus rarity	fan-out (sFlow: sampled)
f. Exfiltration	out/in byte ratio; cumulative egress (CUSUM) against the host's own baseline; rare destination	egress-baseline substitute (Fig. 3)	byte counts plus baseline

Table 3 — The contract, per threat class. The two right-hand columns are what survives each kind of one-way loss.

3.4 What the tap must see, and what is never touched

The sensor never decrypts anything, and it never stores payload. It reads exactly three plaintext things and then discards the packet contents:

- **TLS:** the plaintext ClientHello, to compute the JA4 fingerprint and read the server name (SNI). With Encrypted ClientHello (ECH), the outer ClientHello still yields JA4, but the real server name is hidden, so C2 pairs fall back to being keyed by IP address.
- **QUIC:** packet sizes and timing only. Initial packets are not decrypted, even though their keys are derivable, and streams are not reassembled.
- **DNS:** the query name and record type. Encrypted DNS (DoH/DoT) is treated as encrypted traffic, since names are hidden.

If any of these is missing on a flow, the alert says so. The header-only rule is deliberate: headers are enough to prove who talked to whom, when and how much, which is what every alert's evidence needs. Payload-level forensics stays with other enclave tools, and PS constraint (b) rules out anything that reads encrypted content.

3.5 Detectors and the model

Rules. Each class has behaviour rules over the contract features, with thresholds and their reasons documented in the repository. Rules were tuned on three CTU-13 scenarios and tested on two scenarios held out from tuning.

Flow model. A LightGBM classifier [3] scores flows. LightGBM was chosen over scikit-learn's histogram gradient boosting for one reason: its `predict(pred_contrib=True)` call gives exact per-row TreeSHAP values [4], so every model score arrives with the features that drove it, with no extra dependency. An IsolationForest scores novelty alongside it but only corroborates; it never alerts alone.

- **Training data:** header-only CTU-13-Extended captures (5 scenarios): 31,665 malicious and 83,035 benign flows.
- **No shortcut features:** IP addresses, ports and absolute time are excluded, so the model cannot memorise the capture.
- **Validation:** leave one scenario out. Every score comes from a model that never saw that capture. A random row split was run only to show the leak: it scores 99.96% and is rejected.
- **Operating point:** the threshold is set on the pooled out-of-fold benign scores for at most one ML-only false alarm per 10,000 benign flows.
- **Guard rails:** ML-only alerts are capped at MEDIUM. Only rule evidence, alone or confirmed by the model, can raise HIGH.

Confidence. The `confidence` field is an evidence-strength score set by the detector and by agreement between rule and model. It is not claimed to be a calibrated probability. Visibility is kept separate: it is recorded beside confidence in `observability_state` and `substitutions`, so an analyst sees both how strong the evidence is and what the link could see.

3.6 Correlation into campaigns

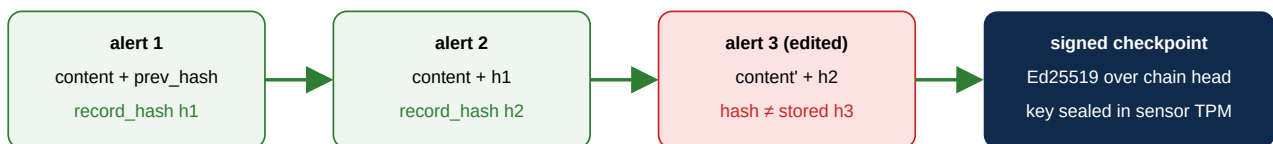
A single alert is a data point; an attack is a story. SAAKSHI joins alerts into a campaign when they share a rare pivot. A pivot counts as rare when few hosts in the enclave's history touch it; shared infrastructure is refused as a pivot, and each join records the pivot used and any pivot refused. The pivots are the internal host, the external server, the JA4 client fingerprint and the queried domain. One known correlator defect: a bare port value (such as TCP port 1) can currently be accepted as a pivot; it is listed in 5.7 and is fixed by restricting pivots to the four types above. In the demo, one internal host's scan, C2 beacon, DGA lookups, DNS tunnel and upload became one campaign, with the ATT&CK stages Discovery, Command and Control, and Exfiltration listed in the order they were observed.

3.7 The alert

Field	PS field?	Definition
timestamp	yes	Capture-clock time (UTC, microseconds) of the last packet in the evidence window — never the time the alert was processed
flow_id	yes	Community ID v1 of the flow, so it joins with Zeek and NetFlow tools. For floods and scans: the one-second window id, with the Community IDs of the top-5 contributing flows listed
threat_class	yes	One of the six PS classes
confidence	yes	Evidence-strength score in [0, 1] (3.5)
evidence	yes	Each feature with its value, its baseline and where the baseline came from
alert_id	extra	UUID for the alert
severity	extra	LOW / MEDIUM / HIGH
observability_state	extra	What the link could see on this flow, e.g. <code>bidirectional</code> or <code>forward_only</code>
substitutions	extra	Each unavailable feature, why, and what replaced it
campaign_id	extra	The campaign this alert joined, with the pivots used and refused
record_hash	extra	SHA-256 link in the alert hash chain

Table 4 — Alert fields. All are present in every sensor-health state. Alerts are written as JSONL and exported in signed offline bundles; the read-only API serves the same records inside the enclave.

3.8 Evidence integrity



verify-log recomputes every hash and names the first record that no longer matches: "BROKEN at index 3 of 10".

Figure 4 — The alert hash chain. Changing one byte of any stored alert breaks the chain at that record.

- Every alert is appended to a SHA-256 hash chain in SQLite. `sih26145 verify-log` recomputes the chain and names the first edited record; Figure 10 shows a real run.
- Chain checkpoints are signed with Ed25519 using a checkpoint key sealed in the sensor's TPM (design; Ed25519 signing is built and used for bundles today). It is separate from the update-signing key, which stays offline, so the sensor can sign checkpoints without ever holding the key that signs software.
- The export is a fixed-size signed bundle for offline media. It contains the alerts (JSONL), header-only records of alerted flows, the chain head, a manifest, and a data sheet that supports the certificate required under Section 63 of the Bharatiya Sakshya Adhiniyam, 2023 [9]. It supports the custody workflow; admissibility remains a matter for the court. Export is refused if the chain is broken.

3.9 Operating safely inside the enclave

No route back. Four independent layers each prevent the sensor from becoming a transmit path:

- Design: the capture runs in its own receive-ring process under a seccomp filter that denies every send system call, and the capture NIC has no IP address. The prototype reads capture files.

- The analysis process opens no network socket. A runtime audit-hook test proves this on every build, and `strace` confirms it (5.4).
- The dashboard runs on a separate management interface with no route to the capture side.
- The hardware diode is the physical guarantee. The deployment acceptance test is that the capture NIC's transmit counter does not change over a run.

Offline by design. The sensor uses no internet, DNS, NTP, certificate checks or threat feeds at runtime; it was tested with networking removed. Updates arrive only as Ed25519-signed offline bundles, which are verified before anything loads. Models are stored as skops files (untrusted types refused at load) or LightGBM text files, never pickle, and every bundle's signature is checked before anything is deserialised.

Time. Timestamps come from the capture NIC. A GPS receiver inside the enclave provides reference time without any network, and a step between the GPS clock and the capture clock is flagged in sensor health.

Baselines. Baselines start from a signed reference, may drift only within bounds and are checked against peer hosts. They use a robust z-score (median/MAD) with a variance floor, so a normally quiet host cannot produce an extreme score from one ordinary transfer. Periods under alert are never learnt as normal, which resists slow poisoning by an attacker who tries to teach the sensor that the attack is normal.



Figure 5 — Sensor health. "No alert" never means "all clear" unless the sensor could actually see.

Overload and flood-as-cover. Under overload the fast lane samples 1-in-N and stamps N on its output, and health moves to DEGRADED. An attacker might flood the link to hide an upload started at the same moment. The flow table therefore keeps a reserved share of admission for traffic from sources the fast lane has not marked as flood sources, and per-host egress counters keep running in the sampled fast lane. An upload that starts during a flood is still tracked and counted, and its alert carries DEGRADED and N. Health changes are recorded as events, and no threat is claimed for a stretch where the sensor was BLIND.

3.10 End-to-end flow

1. **Capture** — a packet arrives on the receive-only interface; its capture-clock time is recorded.
2. **Fast lane** — one-second counters update; a flood or scan raises a provisional alert within 2 s.
3. **Flow build** — the packet joins its flow; direction and observability are recorded.
4. **Score** — when the flow idles 15 s, or every 60 s for long flows, the contract decides which features apply, and the rules and models score it.
5. **Correlate** — the alert joins a campaign if it shares a rare pivot.
6. **Seal** — the alert, with evidence, observability and substitutions, is appended to the hash chain.
7. **Show and export** — the dashboard shows it; the analyst can export a signed bundle to offline media.

4. Implementation

4.1 Technology choices

Layer	Choice	Why this one
Language	Python 3.13	One language for parsing, features, models and API; the team's strongest; mature networking and ML libraries
Packet parsing	dpkt	Fast, pure-Python header parsing with no capture daemon; profiling shows parsing is the largest single cost, so it had to be lean
Flow model	LightGBM	Fast CPU trees with exact per-row TreeSHAP built in, so every score is explained with no extra dependency
Novelty	scikit-learn IsolationForest	Corroborates unusual flows; never alerts alone
Storage	SQLite (WAL) via aiosqlite	A single file per sensor, no server to run in the enclave; WAL lets the pipeline write while the dashboard reads
API and dashboard	FastAPI (read-only, GET/HEAD only) + React dashboard	Typed responses; the React dashboard bundles every font, library and asset, so it needs no CDN or internet
Integrity	SHA-256 hash chain, Ed25519 signatures	Standard, fast, verifiable by anyone with the public key
Model files	skops; LightGBM text	skops refuses untrusted types at load and the bundle signature is checked first; never pickle
Packaging	uv lockfile	Exact, reproducible dependency versions for an offline install

Table 5 — Stack, with the reason for each choice. Everything is open source, runs on CPU only, and carries no licence fee.

4.2 Module breakdown

Part of the repository	Responsibility
sih26145.ingest	Reading PCAP/pcapng and flow records; capture timestamps; the no-socket guarantee
sih26145.streaming, orchestrator	The streaming pipeline: bounded queue, fast lane, flow lane, batched scoring
feature store and contract	Per-host, per-server and per-pair behaviour in fixed memory; the versioned Unidirectional Feature Contract
detectors	Behaviour rules for the six classes; LightGBM, IsolationForest and DGA models
correlation	Rare-pivot campaigns; refused pivots; observed ATT&CK stages
sih26145.storage.chain	Canonical JSON, hash chain, verify-log, export bundle
dashboard/	The offline analyst dashboard
scripts/	benchmark.py, oneway.py (one-direction replays), shard_scale.py (multi-core), baseline.py (Suricata comparison), demo capture builder
tests/	264 automated tests, including the no-socket audit-hook test

Table 6 — Where each responsibility lives. Full documentation is in the repository's docs/ folder.

4.3 Core logic, illustrated

How a detector uses the contract, reduced to its essentials:

```
def score_exfil(flow, contract, store):
    view = flow.observability                    # bidirectional / forward_only / ...
    if contract.available("outbound_inbound_byte_ratio", view):
        ratio = flow.bytes_out / max(flow.bytes_in, 1)
        return rule_hit(ratio > 50, evidence=[("outbound_inbound_byte_ratio", ratio, 50)])
    # reverse side missing: use the declared substitute and say so
    subs = contract.substitute("outbound_inbound_byte_ratio") # egress_z, rarity, off_hours
    egress_z = store.host(flow.src).egress_z(flow.bytes_out)   # robust z, variance floor
    hit = egress_z > Z_MIN and store.dst_rare(flow.dst)
    return rule_hit(hit, evidence=[("src_egress_bytes_z", egress_z, 0.0), ...],
                    substitutions=[{"unavailable_on_this_flow": "outbound_inbound_byte_ratio",
                                   "substituted_by": subs}])
```

How the chain seals an alert:

```
def append(alert, prev_hash):
    body = canonical_json(alert)                # sorted keys, fixed separators
    record_hash = sha256(prev_hash + body)
    db.insert(alert, record_hash)               # append-only; there is no delete path
    return record_hash                         # verify-log recomputes this for every row
```

4.4 The analyst's view

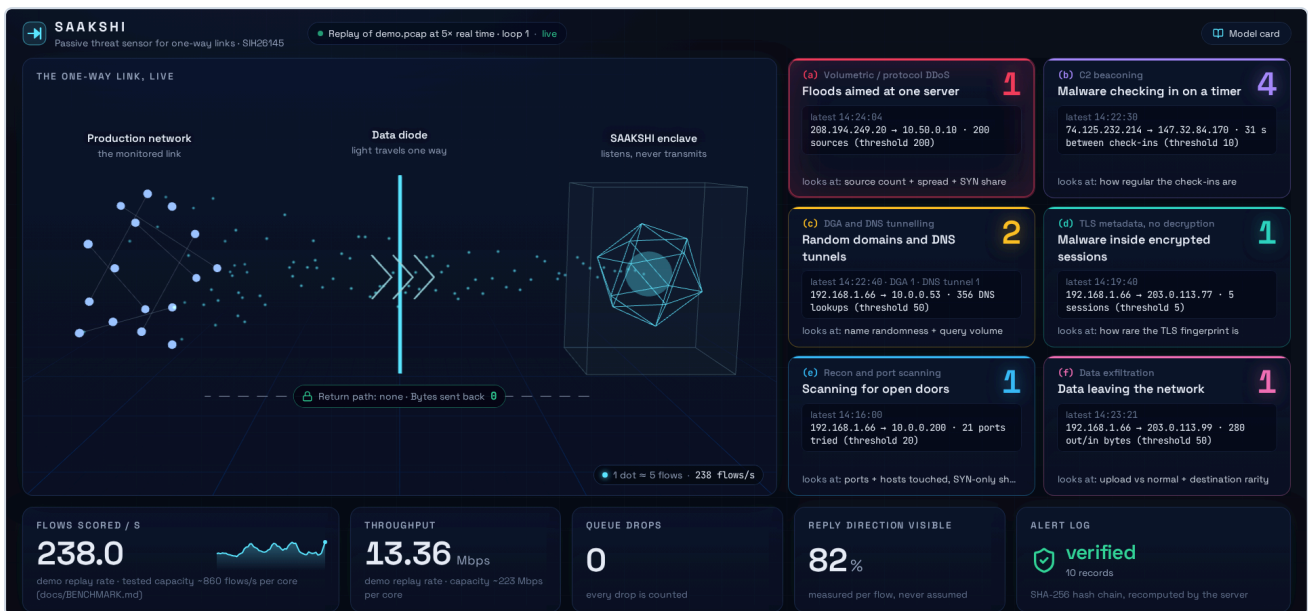


Figure 6 — The live view. The one-way link is drawn explicitly; the tiles show one class each with its evidence. The counters at the bottom include "reply direction visible" and "bytes sent back: 0".

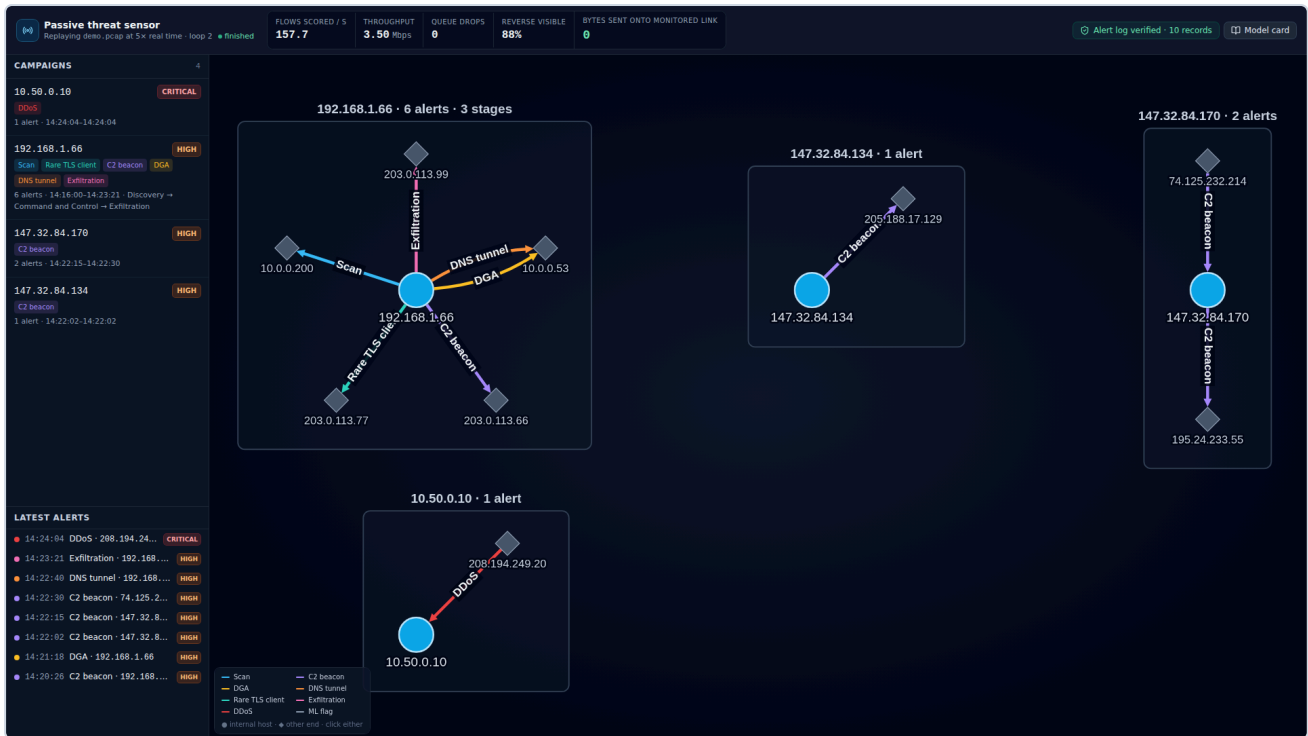


Figure 7 — Campaigns. Captured later in the same replay than Figure 6. One internal host's scan, C2, DGA, DNS tunnel and upload are joined into one campaign; the SYN flood on 10.50.0.10 stands alone.

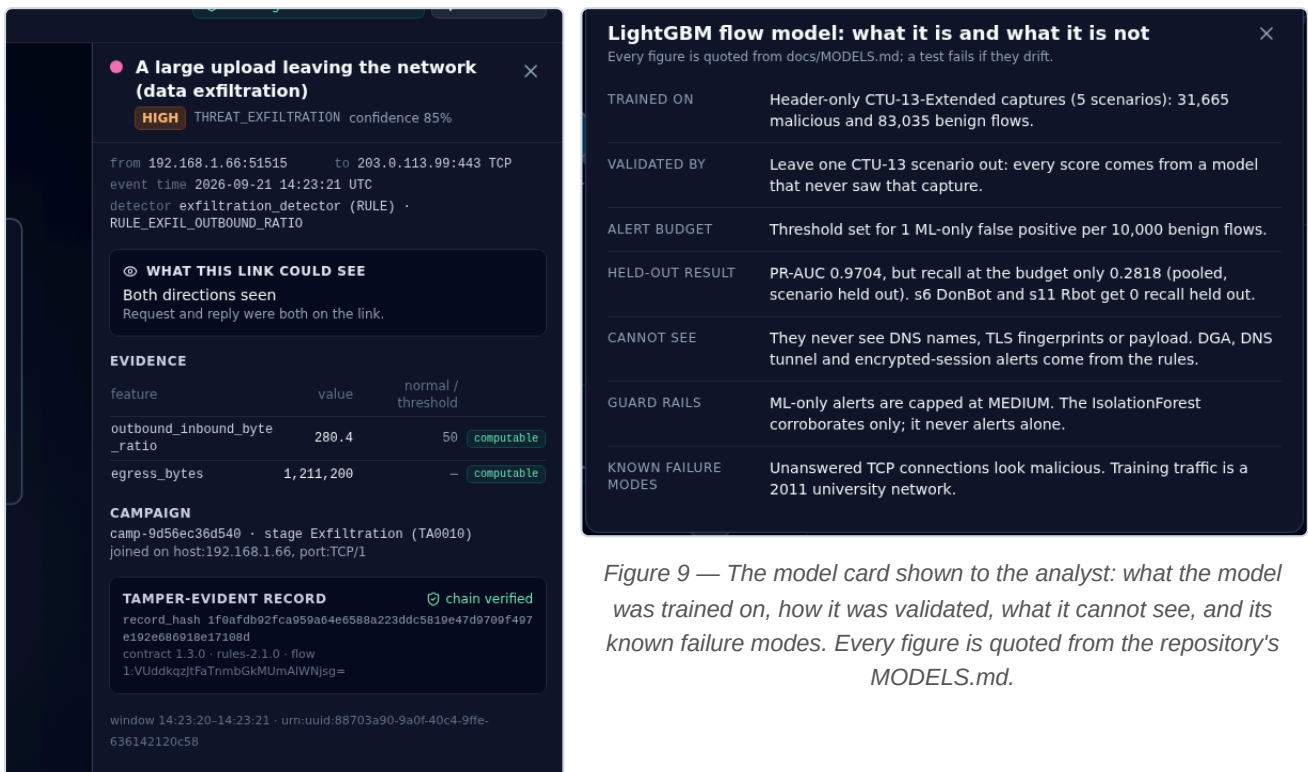


Figure 8 — An alert drawer: "What this link could see", the evidence against its threshold, the campaign, and the tamper-evident record. Screenshot from contract 1.3.0; field layout unchanged in 1.4.0.

Figure 9 — The model card shown to the analyst: what the model was trained on, how it was validated, what it cannot see, and its known failure modes. Every figure is quoted from the repository's MODELS.md.

```

$ sih26145 verify-log --db demo.db
verified: 10 records, chain head 34c179fc8fcb517bbecaaa77c2b7cd629c6f5180cd542fe06a8e019cd77587bd

# one byte of alert #3 changed directly in the database

$ sih26145 verify-log --db demo.db
BROKEN at index 3 of 10: record_hash does not match the record's content

```

Figure 10 — A real run of verify-log: one byte of alert 3 was changed directly in the database, and the log names the broken record.

4.5 Data

Data	Use	Source and licence
CTU-13 and CTU-13-Extended	Real labelled botnet and normal traffic; training, leave-one-scenario-out validation, held-out rule tests, throughput benchmark (scenario 12)	Stratosphere Laboratory, CTU Prague [2]; attribution licence; not redistributed
Generated attacks	SYN, spoofed and UDP-reflection floods; jittered C2 beacons; DGA names; DNS tunnel; rare-JA4 TLS beacon; port and host sweeps; 1.2 MB upload — each injected on real CTU-13 background	The repository's own generator
12 benign stress captures	Silence gate: flash crowd, VoIP, one-way bulk download, failed connections	Generated
Demo capture	11 minutes: every packet of the CTU-13 scenario 5 normal hosts, header-only, with the seven generated attacks	demo/demo.pcap, sha256 in demo/ATTRIBUTION.md

Table 7 — Datasets. Every result in Chapter 5 is labelled REAL or GENERATED by what it rests on.

5. Feasibility and Testing

Every number in this chapter was measured on the prototype and is reproducible from the tagged repository (Appendix B). Where a figure is a design target rather than a measurement, it says so.

5.1 Test machine

All measurements used an Intel Core i5-13450HX laptop (6 performance cores with 2 threads each, and 4 efficiency cores, for 16 threads) under WSL2, on AC power in best-performance mode, with the machine otherwise idle. A native Linux cloud VM produced the identical 91 alerts on CTU-13 scenario 12, confirming detection does not depend on the platform. The no-network run (5.4) reproduced the same 12 alerts on the demo capture.

5.2 Latency

Measure	Result	How it was measured
Fast lane, first attack packet → alert	p99 < 2 s (SYN flood 1.98 s, spoofed flood 1.00 s, UDP reflection 1.50 s, port scan 1.50 s)	Attacks replayed at recorded speed; the clock runs from the first attack packet to the emitted alert
Flow lane, scoring after a flow closes	p99 250 ms	Same benchmark; a flow closes 15 s after its last packet, and long flows are re-scored every 60 s

Table 8 — Latency. The streaming bound for any class is therefore: floods and scans under 2 s, other flows within 15 s of their last packet plus 250 ms, and long flows at every 60 s.

5.3 Throughput and scaling

Series	Cores	flows/s	Mbps	packets/s	Efficiency
Single-core benchmark (idea-deck-v3)	1	~860	~223	~34k	—
Shared-nothing shards (idea-deck-v5), median of 3	1	819	212	~32k	1.00
	2	1,413	366	~56k	0.86
	4	2,426	629	~96k	0.74
	8	3,915	1,015	~155k	0.60
Fast-lane parser ceiling	1	—	—	~108k at 64-byte packets	—

Table 9 — Throughput. Capture: CTU-13 scenario 12 (352,266 packets, 289,266,098 wire bytes, 8,927 flows, 821-byte average packet), replayed from file as fast as the pipeline accepts, with no packet lost. Packets per second for the shard series are derived from the same wall times. In the shard series, the capture was split once, beforehand and untimed, by symmetric flow hash, and N pinned processes each ran one shard. The single-core run in that series (819) is 5% below the earlier series; two things differ and were not separated: it is pinned to one vCPU shared with the database writer, and it records every alert.

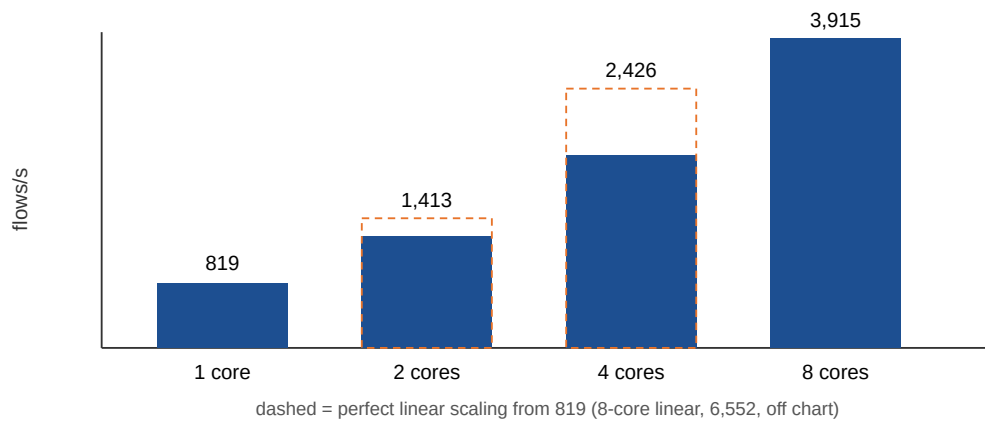


Figure 11 — Measured multi-core scaling: 4.8× on 8 cores, about 1 Gbps at this packet mix.

Why efficiency falls at 8 cores. Two measured effects: the shards are uneven (the busiest carries more than its share), and each core runs slower when all 8 are busy. Plausible causes are shared cache and memory, a lower all-core turbo, and efficiency cores, since the chip has only 6 performance cores. WSL2 does not let us pin to a named core type, so these were not separated.

What the shard run does and does not show. It measures throughput. Because the split was by flow, per-host and per-destination windows were divided between shards, so the sharded runs raised a different alert set from the single process: 73 to 90 alerts against 91. That is why deployment splits by internal host instead (3.2), which keeps each host's history in one shard. The host-split design is what the Grand Finale build will measure.

Sizing. At this packet mix, one 8-core machine handled about 1 Gbps. Minimum-size floods are a different case: 10 Gbps of 64-byte frames is about 14.88 million packets per second, far beyond parsing every packet. The fast lane samples 1-in-N and reports DEGRADED under such floods, which is the designed behaviour, not a failure.

5.4 One-way acceptance

Check	Command	Result
No-socket audit hook	<code>pytest tests/ingest/test_no_outbound_socket.py</code>	2 passed; runs on every build
Run with no network at all	<code>unshare -rn .venv/bin/sih26145 analyze demo/demo.pcap</code>	Same 12 alerts with identical content (sorted content digest equal across runs); only a downed loopback exists; TX 0 → 0
System-call trace	<code>strace -f -e trace=%network -c ...</code>	0 socket, connect or bind calls. No AF_INET, AF_INET6 or AF_PACKET socket created. The only socket activity is one in-process AF_UNIX pair (Python's asyncio wake-ups: 61 one-byte sends, 122 receives)
Deployment acceptance	<code>ip -s link show <capture-if> before and after</code>	Procedure: the capture NIC's TX packet and byte counters must not change

Table 10 — One-way acceptance evidence (docs/ONEWAY-ACCEPTANCE.md). Record hashes differ between any two runs because each alert has a random UUID, so content is compared with that field removed.

5.5 Baseline: Suricata on the same capture

We ran Suricata 8.0.3 with the default ET Open ruleset (53,015 rules, downloaded 29 September 2026), untuned, on the demo capture and its outbound-only version. We ran it twice, once with the exact default

command and once with HOME_NET set to the demo's internal networks. Our attack generator writes fixed TCP sequence numbers, which made Suricata's stream engine reject the generated TCP sessions. To be fair to Suricata, both tools were also run on a copy with valid sequence numbers, and that copy is the main comparison. SAAKSHI's alerts were identical on both copies.

Capture	Tool	ET Open alerts on the 7 attacks	Attacks caught
Both directions	Suricata + ET Open (exact and matched HOME_NET)	0	0 / 7
	SAAKSHI	—	7 / 7 (recon, C2, TLS, DGA, tunnel, exfil, flood)
Outbound only	Suricata + ET Open	0	0
	SAAKSHI	—	3 / 3 attacks that have outbound packets (C2, TLS, exfil)

Table 11 — Baseline (docs/BASELINE.md). Suricata also logged about 97,000 "truncated packet" decoder events on the header-only background; they carry no addresses and are not alerts on hosts. Zeek was not run within the time available.

How to read this. The seven attacks are new: no published signature exists for them. Zero from a signature engine is what signatures are expected to do on new attacks. The result shows the gap behaviour detection fills; it does not show SAAKSHI is better than a tuned Suricata deployment. The two are complementary, and the fair head-to-head is field validation on NTRO captures with the PS's own attack tools, both running side by side.

5.6 Detection results per threat class

PS threat	Tested on	Result	One direction only (IN / OUT)
a. DDoS	REAL Rbot ICMP flood · GEN SYN, spoofed, UDP reflection	all detected; < 2 s on generated floods	IN ✓ · OUT n/a (flood inbound)
b. C2 beacon	REAL 5 CTU-13 botnet scenarios, each held out · GEN 20% / 40% jitter	PR-AUC 0.97 pooled; beacons detected	IN ✓ (server replies alone) · OUT ✓
c. DGA / tunnel	GEN DGA names + DNS tunnel on real background	both detected	n/a (internal resolver in demo)
d. Encrypted	GEN rare-JA4 TLS beacon on real background	detected	IN n/a · OUT ✓
e. Recon	REAL held-out CTU-13 s11 + s12 (686,219 flows) · GEN sweeps	scanner caught; no labelled-normal host flagged	n/a (internal target in demo)
f. Exfiltration	REAL held-out CTU-13 s11 + s12 · GEN 1.2 MB upload	upload caught; no labelled-normal host flagged	IN n/a · OUT ✓ (substitute)

Table 12 — Validation per class. "n/a" means the attack's packets never cross in that direction, so no sensor could see them there. One-direction results come from rewriting captures to keep only inbound or only outbound packets (docs/ONEWAY.md).

Silence gate. Twelve benign stress captures (flash crowd, VoIP, one-way bulk download, failed connections) raise zero alerts, and the gate is re-run after every change.

5.7 Operating points and known limits

These are the measured operating limits of the current MVP; they are not deployment guarantees. They are left off the six-slide deck for space and stated here in full for anyone deploying the sensor.

What	Measured	What it means and what we do about it
Flow model recall at the alert budget	PR-AUC 0.9704, but recall at ≤ 1 false alarm per 10k benign flows is 0.2818 (pooled, scenario held out); two scenarios (s6 DonBot, s11 Rbot) get 0 recall when held out	The model ranks well but is strict at a tight budget. This is why it never alerts HIGH alone and why rules carry the classes the model cannot see (DNS names, TLS fingerprints). Field tuning on NTRO captures sets the budget per deployment.
C2 rule precision	0.44	Periodic benign software looks like beaconing. In the demo, 3 of the 10 alerts are C2 alerts on real CTU-13 hosts. Persistence over more beacon cycles and destination rarity are the next refinements.
Outbound-only exfiltration substitute	On the outbound-only demo copy, 94 of SAAKSHI's 100 alerts touched background hosts (91 from the egress-baseline rule)	Without replies, the substitute is noisy on a short capture, because host baselines have little history. The robust z with a variance floor and the signed reference baseline (3.9) are the design answer; the alerts carry <code>forward_only</code> so the analyst knows.
Overload	At 2× the sustainable rate, 5.6% of flows were dropped, all counted	Drops are counted, never silent; this is what DEGRADED exists to report.
Multi-core alert set	Flow-hash shards: 73–90 alerts against 91	Throughput scales; state must not be split. Deployment splits by internal host (5.3).
Native Linux VM throughput	Slower 2016-generation cloud CPU; identical 91 alerts	Detection is portable; throughput depends on the CPU.
Campaign pivot on a bare port	Observed in one run: a join made on TCP port 1	A correlator defect, not a detection result. Pivots are being restricted to host, server, JA4 and domain (3.6).
Training data age	CTU-13 was captured in 2011	Encrypted-traffic features are version-agnostic by design; current TLS 1.3 / QUIC traffic is the first thing field validation measures.

Table 13 — Operating points and limits. All figures are in the repository's docs (*MODELS.md*, *RULES.md*, *BENCHMARK.md*, *BASELINE.md*, *ONEWAY.md*).

5.8 Feasibility on four axes

Axis	Assessment
Technical	Working prototype: all six classes, both lanes, the contract, the chain and the dashboard. Measured throughput, latency and one-way acceptance. 264 automated tests.
Financial	Open-source stack; no licence fee; CPU only, no GPU. Measured about 1 Gbps on one 8-core laptop-class machine at the benchmark packet mix, which indicates a sensor can run on commodity server hardware; production sizing depends on the link and its packet mix.
Operational	Offline install from a locked dependency set; signed offline updates; one read-only dashboard; JSONL output that any offline tool can ingest; Community ID joins with Zeek and NetFlow records.
Deployment fit	Built for exactly the diode enclave the PS describes: receive-only, no runtime network, evidence that leaves only on offline media.

Table 14 — Feasibility.

5.9 Risk register

Risk	Likelihood / impact	Mitigation
Link faster than one sensor	High / medium	Receive-only broker splits by internal host; busy hosts split by host-server pair with merged counters; measured 4.8× on 8 cores
Sensor becomes a path back	Low / critical	Seccomp receive-only capture; 0 network sockets (tested every build); separate management interface; hardware diode; TX-counter acceptance test
Real traffic differs from CTU-13	High / high	Tune on one period of NTRO captures, report on a later one; silence gate after every change; REAL/GENERATED labelling of every result
A flood is used to hide an upload	Medium / high	Flood sources cannot crowd out new flows; per-host egress counters keep running; DEGRADED and N on every alert in the window
Baseline poisoning over weeks	Medium / high	Signed reference; bounded drift; peer checks; periods under alert never learnt
Malicious update bundle	Low / critical	Ed25519 signature verified before anything loads; no pickle; update key kept offline
Evidence tampering after the fact	Low / high	Hash chain with TPM-signed checkpoints; verify-log names the first edited record; export refused on a broken chain
Clock drift in an offline enclave	Medium / medium	Capture-NIC timestamps; GPS reference inside the enclave; steps flagged in sensor health
Encryption hides more over time (ECH, DoH, QUIC)	High / medium	Metadata-only features; declared fallbacks (C2 keyed by IP under ECH; DoH treated as encrypted); the alert states what was hidden

Table 15 — Risk register.

6. Results and Impact

6.1 What the system produces



Figure 12 — The demo capture: 11 minutes of real CTU-13 background with 7 generated attacks. All 7 attacks alerted.

An analyst facing this capture with flow logs alone has 20,857 records. With SAAKSHI they read 10 alerts and 2 early flood flags, grouped into 4 campaigns. Early flags are provisional fast-lane alerts that the flow lane then confirms. Each alert already carries its evidence against normal, what the link could see, and its place in the chain. We state the counts as measured rather than converting them into analyst-minutes, because a minutes-per-alert figure would have to be assumed.

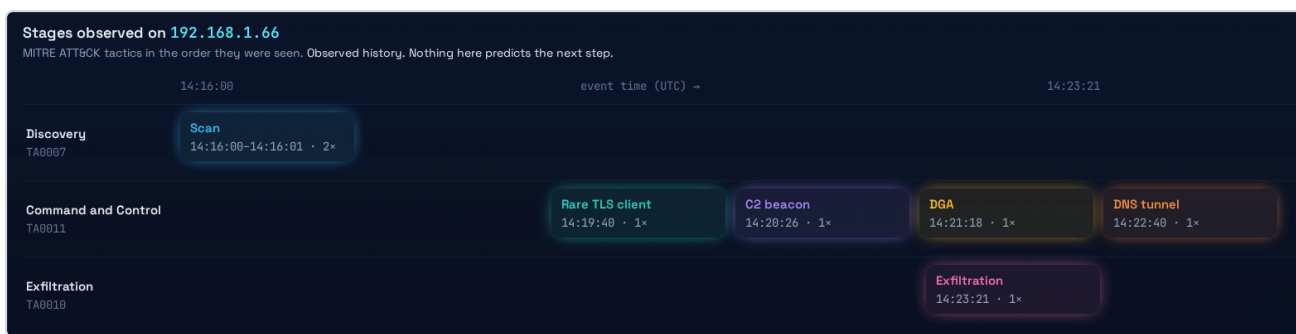


Figure 13 — The campaign view for host 192.168.1.66: scan, then rare TLS client and C2 beacon, then DGA and DNS tunnel, then upload. This is observed history, not a prediction.

6.2 Before and after

Question an analyst asks	Flow logs alone	With SAAKSHI
What do I read first?	20,857 flow records	4 campaigns, 10 alerts, 2 early flags
Why did it alert?	Reconstruct from raw fields	Evidence value against the host's normal, on the alert
Could the link see the reply?	Not recorded	observability_state and substitutions on every alert
Has anyone edited this record?	Depends on log storage	verify-log names the first edited record
How fast was a flood flagged?	When someone queries	Under 2 s from the first attack packet (p99)

Table 16 — Before and after, on the demo capture.

6.3 Who benefits

- **NTRO and NCIPC analysts** get explained alerts inside the enclave, each saying what the link could see.
- **Critical-infrastructure operators** (power, telecom, banking, rail) get detection on links behind a diode without opening any path back. OT protocols crossing the gateway are flagged.

- **CERT-In and CSIRTs** get an evidence bundle with detectable edits, and a data sheet that supports a Section 63 certificate.

Benefit	How
Social	Safer power, telecom, banking and rail links, the services citizens depend on
Economic	Open source, no licence fee, commodity CPUs
Environmental	CPU only; no GPU needed to train or run
Operational	10 alerts to read, not 20,857 flows; evidence ready for custody

Table 17 — Benefits.

7. Conclusion, Prototype Status and Future Scope

7.1 Conclusion

SAAKSHI treats a one-way link as a change in what counts as evidence, not only as a networking constraint. The Unidirectional Feature Contract means a detector never silently uses a signal the link did not carry. Every alert says what was seen, what was missing and what replaced it. Two lanes give floods and scans a sub-two-second path while everything else is scored with a stated bound. The hash chain, signed checkpoints and offline export keep the evidence fit for custody. The sensor's no-send behaviour is measured and reproducible; deployment acceptance additionally checks that the capture NIC's transmit counter stays unchanged across a run.

7.2 Prototype status

Component	Status
Ingest (PCAP/pcapng), fast lane, flow lane, feature store	BUILT, MEASURED
Unidirectional Feature Contract with substitutions and observability state	BUILT, MEASURED (real forward-only alert, Appendix A)
Rules for all six classes; LightGBM, IsolationForest, DGA model	BUILT, MEASURED
Rare-pivot campaigns; observed ATT&CK stages	BUILT
Hash chain, verify-log, export bundle with Section 63 data sheet	BUILT, MEASURED
Signed offline update bundles (Ed25519, skops)	BUILT
No-socket guarantee; run with no network	BUILT, MEASURED
Offline dashboard and read-only API	BUILT (live replay at the link in Appendix C)
Multi-core shards	MEASURED (flow-hash split); host split is design
Sensor health NORMAL / DEGRADED / BLIND	DESIGN (drops are already counted)
TPM-sealed checkpoint signing	DESIGN (Ed25519 signing exists for bundles)
Seccomp receive-ring capture on a live NIC; GPS time	DESIGN
NetFlow/IPFIX ingest; flow-record-only features	DESIGN (the contract already declares their status)
Robust, floored, signed baselines; flood-admission reserve	DESIGN

Table 18 — What is built and measured today, and what the three months to the Grand Finale complete.

7.3 Field validation plan (on NTRO-provided captures)

1. **Benign load** (iperf3, TRex) and rare benign TLS clients: false alarms per analyst-hour, per class.
2. **Attacks** with the PS's own tools (hping3, Slowloris, dnscat2, iodine, DGA families, a C2 emulator): recall at a fixed alert budget, per class and per direction.
3. **Held-out periods**: thresholds tuned on one period of captures and reported on a later one, never the same data.
4. **Side by side** with the operator's tuned signature stack, on the same captures.
5. **Acceptance**: one offline command replays the captures inside the enclave, with the same one-way checks (0 sockets, capture-NIC TX counter unchanged).

7.4 Future scope

- Host-keyed sharding with hot-host splitting, then re-measure scaling and alert parity.
- Sensor-health states wired to drops, gaps, lost direction and rate collapse.
- A quantified one-way degradation matrix: each attack under both directions, inbound only, outbound only, reply loss, overload and flow-records-only.
- Persistence scoring for C2 over more beacon cycles; host-level DNS aggregation; long-window low-and-slow exfiltration tests.
- A Zeek baseline alongside Suricata; an IPv6 parity test.
- A sustained 6–24 h run: memory is bounded by design (flow table and window limits), disk by the rolling buffer and export rotation; the run measures both.
- Restrict campaign pivots to host, server, JA4 and domain (fixes the bare-port join in 5.7).
- A NetFlow/IPFIX adapter so flow-records-only links run through the same contract.
- Detector, rule, contract and threshold versions carried in every exported alert (already present as `contract_version` and `model_version`) and in the bundle manifest.
- Fix the attack generator's fixed TCP sequence numbers so every tool can reassemble the demo sessions.

References

1. National Technical Research Organisation, "SIH26145: AI-Based Detection of Cyber Threats in Unidirectional IP Traffic," Smart India Hackathon 2026 problem statement.
2. S. García, M. Grill, J. Stiborek and A. Zunino, "An empirical comparison of botnet detection methods," *Computers & Security*, vol. 45, pp. 100–123, 2014. (CTU-13 dataset, Stratosphere Laboratory, CTU Prague.)
3. G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
4. S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, pp. 56–67, 2020.
5. B. Anderson and D. McGrew, "Identifying Encrypted Malware Traffic with Contextual Flow Data," in *Proc. ACM Workshop on Artificial Intelligence and Security (AISec)*, 2016.
6. FoxIO, "JA4+ Network Fingerprinting," technical specification.
7. M. Antonakakis et al., "From Throw-Away Traffic to Bots: Detecting the Rise of DGA-Based Malware," in *Proc. USENIX Security Symposium*, 2012.
8. MITRE, "ATT&CK for Enterprise — Tactics."
9. Government of India, *The Bharatiya Sakshya Adhiniyam, 2023*, Section 63.
10. Open Information Security Foundation, "Suricata" (version 8.0.3), with the Proofpoint Emerging Threats Open ruleset.
11. Corelight, "Community ID Flow Hashing" specification.

Appendix A — The full alert, field by field

A real alert from the prototype: the demo capture replayed with only outbound packets. Abridged only where marked.

```
{ "alert_id": "urn:uuid:0d4f9d49-4691-4b7c-b396-ec66ca3c2cf5",
  "timestamp": "2026-09-21T14:23:21.997499+00:00",
  "flow_id": "1:VUddkqzJtFaTnmbGkMUmAlWNjsg=",
  "threat_class": "THREAT_EXFILTRATION", "severity": "HIGH", "confidence": 0.85,
  "evidence": [
    {"feature": "src_egress_bytes_z", "value": 720.88, "baseline": 0.0, "baseline_source":
"host_ewma"},
    {"feature": "dst_distinct_srcs_longterm", "value": 1.0},
    {"feature": "off_hours", "value": true},
    {"feature": "egress_bytes", "value": 1211200} ],
  "observability_state": "forward_only",
  "substitutions": [{ "unavailable_on_this_flow": "outbound_inbound_byte_ratio",
    "reason": "reverse direction not observed on this flow (forward_only)",
    "substituted_by": ["src_egress_bytes_z", "dst_distinct_srcs_longterm", "off_hours"] }],
  "contract_version": "1.4.0", "model_version": "rules-2.1.0",
  "campaign_id": "camp-ee0de4fb7d3b", "host_stage": "TA0010",
  "detector": {"name": "exfiltration_detector", "type": "RULE", "version": "2.1.0"},
  "detection": { "rule_matches": ["RULE_EXFIL_EGRESS_BASELINE"],
    "correlation": {"joined_on": ["host:192.168.1.66", "port:TCP/1"], "refused_pivots": []}
  },
  "flow": {"src_ip": "192.168.1.66", "dst_ip": "203.0.113.99", "dst_port": 443, "protocol":
"TCP",
    "window_start": "2026-09-21T14:23:20+00:00", "window_end": "2026-09-
21T14:23:21.997499+00:00"},
  "feature_summary": {"total_packets": 800, "total_bytes": 1211200},
  "record_hash": "56e30baa8b3bef29e0cfc66efb648cbae301236e81401d243502dbcd4f57a260" }
```

Full files: docs/examples/alert-exfil-forward-only.json and, for the same upload with both directions seen, docs/examples/alert-exfil.json.

Appendix B — Reproducing every number

```
git clone https://github.com/SIH-2026-CSM-A/26145 && cd 26145
git checkout idea-deck-v5 # v3: single-core benchmark · v4: fast lane,
one-way replay · v5: scaling, baseline, acceptance
uv sync
uv run pytest # 264 tests, incl. the no-socket audit hook
uv run sih26145 analyze demo/demo.pcap --db demo.db
uv run sih26145 verify-log --db demo.db # hash-chain check
uv run python scripts/oneway.py rewrite OUT demo/demo.pcap out.pcap # one-direction
replay
uv run python scripts/benchmark.py CAPTURE.pcap # throughput and latency
uv run python scripts/shard_scale.py split 8 && uv run python scripts/shard_scale.py run 8 1
uv run python scripts/baseline.py run && uv run python scripts/baseline.py report
unshare -rn .venv/bin/sih26145 analyze demo/demo.pcap --db /tmp/nonet.db # no network at all
```

CTU-13 data is not redistributed; download it from the Stratosphere Laboratory, CTU Prague. The demo analysis needs SIH26145_INTERNAL_CIDRS set to the demo's internal networks, as documented in the README and ISOLATION.md.

Appendix C — Links

Artefact	Link
Source code and documentation	github.com/SIH-2026-CSM-A/26145
Replay demo (evaluation copy, read-only)	8.234.101.133.sslip.io
Narrated demo film (5 min)	drive.google.com/file/d/1YLhOLiwpq-k279OFIXXU5RgD0NXUCW4I/view
Measurement documents	docs/BENCHMARK.md · docs/BASELINE.md · docs/ONEWAY.md · docs/ONEWAY-ACCEPTANCE.md · docs/MODELS.md · docs/EXPORT.md

Appendix D — Glossary

Term	Meaning
Data diode	Hardware that lets data travel in one direction only
Monitoring enclave	The isolated network where the copied traffic is analysed
Flow	All packets between two endpoints on one protocol and port pair, in both directions where seen
JA4	A fingerprint of the plaintext TLS ClientHello; identifies the client software without decrypting anything
DGA	Domain generation algorithm: malware that invents new domain names to reach its controller
C2 beacon	Regular check-ins from infected software to its command-and-control server
CUSUM	Cumulative sum: a running total that grows when a value stays above normal, so slow, steady exfiltration adds up
TreeSHAP	An exact method that says how much each feature pushed a tree model's score up or down
Community ID	A standard hash of a flow's endpoints, so the same flow has the same ID in different tools
PR-AUC	Area under the precision–recall curve; how well a model ranks attacks above normal traffic
Observability state	What the link could see on a flow, e.g. both directions or forward only